

```
0000 WITH UNCHECKED_DEALLOCATION;WITH LEX,-GRMR_OPS;SEPARATE(-IDL)IF--|
--|PROCEDURE-READ_GRMR(-NOM_TEXTE :STRING-) ISIF IFILE, OFILE: FILE_TYPE;FICHER GRAMMAIRE ET TEXTE D'IN
ITALISATIONS SYMBOLES OPERATEURS;TER_COUNT: NATURAL:= 0;ALT_COUNT: NATURAL:= 0;SEMAN_COUNT: INTEGER:= 0;NOMBRE DE SYLLAB
ES SEMANTIQUES SEMAN_ALT_COUNT: INTEGER:= 0;NOMBRE D'ALTS AVEC SEMANTIQUE SOURCE_LIST: SEQ_TYPE;LISTE-DES SOURCELINES
SOURCEPOS: TREE;LA SOURCE_POSITION ARITY_TABLE: ARRAY (0..300) OF INTEGER:= (OTHERS => -1);
--|PACKAGE-LALR_LEX IS
--|PROCEDURE AVANCER;FUNCTION TOKEN-RETURN STRING;
--|END LALR_LEX;
--|PACKAGE-BODY LALR_LEX IS
0010 SLINER: STRING(1..256);LIGNE-COURANTE LINE_COUNT: NATURAL:= 0;NOMBRE DE LIGNES VUES LINE_TAKEN: NAT
L:= 0;N DE LA DERNIERE LIGNE PRISE DANS UN SOURCE_LINE COLR: NATURAL:= 1;PROCHAINE COLONNE BALAYER LAST: NATU
L:= 0;NB DE-CARACTERES DANS LA LIGNE TS, TER: NATURAL;BORNES DU LEXEME
--|PROCEDURE-AVANCER PROCEDURE AVANCER IS
SOURCE_LINE: TREE;LE SOURCELIN
E COURANT BEGIN<<START_GET>>IF COL > LAST-THEN IF END_OF_FILE (IFILE) THEN TS := SLINERFIRST;TE := SLINERFIRST+3;SLINER
SLINERFIRST..SLINERFIRST+3 := "end";ELSE GET_LINE(-IFILE, SLINER, LAST) := SLINERCOUNT := LINE_COUNT + 1;COL := 1;END IF;
END-IF;WHILE COL < LAST AND THEN (SLINER(COL) = ' ' OR ELSE SLINER(COL) = ASCII.HT) LOOP COL := COL + 1;END-LOOP;IF
COL < LAST-THEN IF SLINER(COL..COL+1) = "-" OR SLINER(COL..COL+1) = "/" OR SLINER(COL..COL+1) = "##" THEN COL := LAST + 1;GOTO START_G
ET;END IF;ELSEIF COL > LAST THEN LIGNE-BLANCHE GOTO START_GET;END-IF;TS := COL;WHILE COL < LAST L
OOP EXIT WHEN SLINER(COL) = ' ' OR-ELSE SLINER(COL) = ASCII.HT;IF COL < LAST AND THEN SLINER(COL..COL+1) = "-" THEN COL := LAST
+ 1;GOTO START_GET;END IF;COL := COL + 1;END-LOOP;TE := COL - 1;IF LINE_COUNT = LINE_TAKEN THEN SOU
RCE_LINE := MAKE(DN_SOURCELIN);DI (XD_NUMBER, SOURCE_LINE, LINE_COUNT);LIST(SOURCE_LINE, (TREE_NIL, TREE_NIL));SOURC
E_LIST := APPEND(SOURCE_LIST, SOURCE_LINE);END-IF;SOURCEPOS := MAKE_SOURCE_POSITION(SOURCE_LINE, SRCOL_IDX(TS));END AVANCER;
--|FUNCTION TOKEN FUNCTION TOKEN RETURN
STRING IS BEGIN RETURN SLINER(TS..TE);END;
--|PROCEDURE-LOAD_DIANA PROCEDURE LOAD_DIANA IS OP: TREE;SYM: TREE;NODE_POS: NATURAL;DIANATBL_FILE: TEXT_IO.FILE_TYPE;
BUFFER: STRING(1..127);LAST: NATURAL-RANGE 0..127;COLR: NATURAL-RANGE 0..127;PROCEDURE SKIP_BLANKS IS BEGIN WHILE CO
L < LAST AND THEN (BUFFER(COL) = ' ' OR BUFFER(COL) = ASCII.HT) LOOP COL := COL + 1;END-LOOP;PROCEDURE FIND_BLANK I
S BEGIN WHILE COL < LAST AND THEN (BUFFER(COL) = ' ' AND THEN (BUFFER(COL) = ASCII.HT) LOOP COL := COL + 1;END-LOOP;
END;BEGIN TEXT_IO.OPEN(DIANATBL_FILE, TEXT_IO.IN_FILE, "/idl_tools/diana.tbl");TROUVER CLASSE ALL_SOURCE;LOOP GET_LINE(DIAN
ATBL_FILE, BUFFER, LAST);IF LAST > 0 THEN IF BUFFER(1) = 'C' THEN COL := 2;SKIP_BLANKS;EXIT WHEN-BUFFER(COL..LAST) = "ALL_SOU
RCE";END IF;END-IF;END LOOP TROUVER CLASSE ALL_SOURCE;TRAITER TOUTE CLASSE ALL_SOURCE;LOOP GET_LINE(DIANATBL_FILE,
BUFFER, LAST);IF LAST > 0 THEN IF BUFFER(1) = 'E' THEN FIN DE CLASSE ALL_SOURCE;EXIT WHEN-BUFFER(COL..LAST)
= "ALL_SOURCE";FIN DE CLASSE SOURCE_NAME: FIN DE TRAITEMENT;ELSEIF BUFFER(1) = 'N' THEN POUR UN NOEUD COL := 2;SKIP
BLANKS;FIND_BLANK;POUR PASSER SUR-LE NUMERO-DE NOEUD-SKIP_BLANKS;OP := MAKE(DN_SEM_OP);CREER-UN NOEUD SEM_OP;NODE_POS
:= INTEGER-VALUE(BUFFER(2..COL-1));NUMERO DE NOEUD-DI(XD_SEM_OP, OP, NODE_POS);OP_XD_SEM_OP := POS;SYM := STORE_SYM;POU
R COL..LAST;NOM DU NOEUD-LIST(SYM, INSERT(LIST(SYM), OP));ARITY_TABLE(NODE_POS) := 0;ELSEIF BUFFER(1) = 'A' OR ELS
E-BUFFER(1) = 'B' OR ELSE BUFFER(1) = 'I' THEN COL := 2;SKIP_BLANKS;FIND_BLANK;SKIP_BLANKS;IF COL + 2 <= LAST AND THEN BUFFER(COL..
COL+2) = "as" THEN ATTRIBUT as_xxx COL := 2;ATTRIBUT RECULER AU DEVAUT DU NUMERO D ATTRIBUT SKIP_BLANKS;IF BUFFER(COL) = 'I' THEN
NUMERO NEGATIF INDIQUE SEQUENCE ARITY_TABLE(NODE_POS) := 4;ELSE ARITY_TABLE(NODE_POS) := ARITY_TABLE(NODE_POS) + 1;
END IF;END-IF;END-IF;END LOOP TRAITER TOUTE CLASSE ALL_SOURCE;TEXT_IO.CLOSE(DIANATBL_FILE);EXCEPTION WHE
N N_ERROR => TEXT_IO.CLOSE(DIANATBL_FILE);END LOAD_DIANA;
--|PROCEDURE-LOAD_TERMINALS PROCEDURE LOAD_TERMINALS IS TER: TREE;SYM: TREE;DEFLIST: SEQ_TYPE;USE L
EX;BEGIN FOR T-IN LEX_TYPE LOOP TER := MAKE(DN_TERMINAL);SYM := STORE_SYM(LEX.LEX_IMAGE(T));DEFLIST := LIST(SYM);
WHILE NOT IS_EMPTY(DEFLIST) AND-THEN HEAD(DEFLIST).TY = DN_TERMINAL LOOP DEFLIST := TAIL(DEFLIST);END-LOOP;IF NOT I
S_EMPTY(DEFLIST) THEN PUT("***DUPLICATE TERMINAL IMAGE - ");PUT_LINE(LEX_IMAGE(T));END-IF;LIST(SYM, INSERT(
LIST(SYM), TER));DI(XD_SYMREP, TER, SYM);DI(-XD_TER_NBR, TER, LEX_TYPE_POS(T));END LOOP;END LOAD_TERMINALS;
--|PROCEDURE-PROCESS_GRAMMAR PROCEDURE PROCESS_GRAMMAR IS
USER_ROOT: TREE;GRAMMAR: TREE;RULE: TREE;ALTERNATIVE: TREE;SYLLABLE: TREE;SYMBOL: TREE;SEQ: SEQ_TYPE;
RULE_LIST: SEQ_TYPE:= (TREE_NIL, TREE_NIL);ALT_LIST: SEQ_TYPE;SYL_LIST: SEQ_TYPE;SEMAN_LIST: SEQ_TYPE;SEM_S: TREE;
--|PROCEDURE-MAKE_RULE PROCEDURE MAKE_RULE (TEXT
:STRING) IS DEFLIST: SEQ_TYPE;BEGIN IF TEXT = "" OR TEXT = "==" THEN RAISE PROGRAM_ERROR;END IF;SYMBOL := STORE_SYM(
TEXT);PUT_LINE("RULE = " & TEXT);RULE := MAKE(DN_RULE);DI(XD_NAME, RULE, SYMBOL);DI(LX_SRCPOS, RULE, SOURCEPOS);
RULE_LIST := APPEND(RULE_LIST, RULE);SEQ := LIST(SYMBOL);DEFLIST := SEQ;WHILE NOT IS_EMPTY(DEFLIST) AND THEN HEAD(D
EFLIST).TY = DN_RULE AND THEN HEAD(DEFLIST).TY = DN_TERMINAL LOOP DEFLIST := TAIL(DEFLIST);END-LOOP;IF NOT IS_E
MPTY(DEFLIST) THEN ERROR(SOURCEPOS, "DUPLICATE RULE - " & TEXT);END-IF;LIST(SYMBOL, APPEND(SEQ, RULE));ALT_LIST :=
(TREE_NIL, TREE_NIL);END MAKE_RULE;
--|PROCEDURE-MAKE_SYLLABLE PROCEDURE MAKE_SYLLABLE (TEXT:STRING) IS BEGIN IF TEX
T = "" OR TEXT = "==" THEN RAISE PROGRAM_ERROR;END IF;IF TEXT = "" THEN SYMBOL := STORE_SYM("");ELSE SYMBOL :=
STORE_SYM(TEXT);END-IF;SEQ := LIST(SYMBOL);WHILE NOT IS_EMPTY(SEQ) AND THEN HEAD(SEQ).TY = DN_TERMINAL AND THEN
HEAD(SEQ).TY = DN_RULE LOOP SEQ := TAIL(SEQ);END-LOOP;IF IS_EMPTY(SEQ) OR ELSE HEAD(SEQ).TY = DN_TERMINAL THEN
SYLLABLE := MAKE(DN_NONTERMINAL);ELSE SYLLABLE := MAKE(DN_TERMINAL);DI(XD_TER_NBR, SYLLABLE, DI(XD_TER_NBR, HEAD(
SEQ)));END-IF;DI(XD_SYMREP, SYLLABLE, SYM);DI(LX_SRCPOS, SYLLABLE, SOURCEPOS);SYL_LIST := APPEND(SYL_LIST, SYL
LABLE);END MAKE_SYLLABLE;
--|PROCEDURE-MAKE_SEMANTICS_GET_TOKEN PROCEDURE MAKE_SEMANTICS_GET_TOKEN (IN_TEXT:STRING) IS TEXT: CONSTANT STRING:= IN_TEXT;COPY OF THE ARG
UMENTS USE-GRMR_OPS;SEM_OP: GRMR_OP;SEMAN_SYM: TREE;NODE_NAME_POS: INTEGER;DEFLIST: SEQ_TYPE;SEMAN:
TREE;NODE_NAME_ARY: ARITIES;BEGIN SEM_OP := GRMR_OP.VALUE(TEXT);AVANCER;CASE SEM_OP IS WHEN G_ERROR => IF SEMAN_SYM.TY =
DN_VOID THEN DEFLIST := (TREE_NIL, TREE_NIL);ELSE DEFLIST := LIST(SEMAN_SYM);WHILE NOT IS_EMPTY(DEFLIST) AND THEN HEAD(DEFLIST).TY =
DN_SEM_OP LOOP DEFLIST := TAIL(DEFLIST);END-LOOP;IF IS_EMPTY(DEFLIST) THEN ERROR(SOURCEPOS, "NAME NOT FOUND
AFTER - " & TEXT);ELSE NODE_NAME_POS := DI(XD_SEM_OP, HEAD(DEFLIST));AVANCER;SEMAN := MAKE(DN_SEM_NODE);DI(XD_SEM_OP, SEMAN, G
RMR_OP_POS(SEM_OP));DI(XD_KIND, SEMAN, NODE_NAME_POS);SEMAN_LIST := APPEND(SEMAN_LIST, SEMAN);SEMAN_COUNT := SEMAN_COUNT + 1;NODE_NA
ME_ARY := ARITIES(VAL(ARITY_TABLE(NODE_NAME_POS)));CASE SEM_OP IS WHEN N_0..N_DEF => IF NODE_NAME_ARY = NULLARY THEN ERROR(SOURC
EPOS, "NODE MUST BE NULLARY - " & TEXT & " " & PRINT_NAME(SEMAN_SYM));END IF;WHEN N_1 => IF NODE_NAME_ARY = UNARY THEN ERROR(SOURC
EPOS, "NODE MUST BE UNARY - " & TEXT & " " & PRINT_NAME(SEMAN_SYM));END IF;WHEN N_2..N_V2 => IF NODE_NAME_ARY = BINARY THEN ERR
OR(SOURCEPOS, "NODE MUST BE BINARY - " & TEXT & " " & PRINT_NAME(SEMAN_SYM));END IF;WHEN N_3..N_V3 => IF NODE_NAME_ARY = TERNARY THEN ERR
OR(SOURCEPOS, "NODE MUST BE TERNARY - " & TEXT & " " & PRINT_NAME(SEMAN_SYM));END IF;WHEN N_4..N_V4 => IF NODE_NAME_ARY = ARBITR
ARY THEN ERROR(SOURCEPOS, "NODE MUST BE ARBITRARY - " & TEXT & " " & PRINT_NAME(SEMAN_SYM));END IF;WHEN OTHERS => RAISE PROGRAM_ERROR
END-IF;WHEN G_INFIX | G_UNARY => IF TOKEN(TOKENFIRST) = " " THEN ERROR(SOURCEPOS, "QUOTED STRING REQUIRED AF
TER - " & TEXT & " (TOKEN = " & TOKEN & ")");ELSE SEMAN := MAKE(DN_SEM_NODE);DI(XD_SEM_OP, SEMAN, GRMR_OP_POS(SEM_OP));DECLAREM
SYM: TREE:= STORE_SYM(TOKEN);BEGIN DI(XD_KIND, SEMAN, SYM);PUT_LINE(OFILE, "STORE_SYM (" & TOKEN & ")");SET_OUTPUT(OFILE);
PRINT_TREE(SYM);SET_OUTPUT(STANDARD_OUTPUT);END-IF;SEMAN_LIST := APPEND(SEMAN_LIST, SEMAN);SEMAN_COUNT := SEMAN_COUNT + 1;AVANCER;
END-IF;WHEN OTHERS => SEMAN := MAKE(DN_SEM_OP);DI(XD_SEM_OP, SEMAN, GRMR_OP_POS(SEM_OP));SEMAN_LIST := APPEN
D(SEMAN_LIST, SEMAN);SEMAN_COUNT := SEMAN_COUNT + 1;END-IF;END MAKE_SEMANTICS_GET_TOKEN;
--|PROCEDURE-MAKE_TERMINAL PROCEDURE MAKE_TERMINAL (-TEXT:STRING) IS
SYMBOL: TREE;DEFLIST: SEQ_TYPE;BEGIN IF TEXT = "" THEN SYMBOL := FIND_SYM("");ELSE SYMBOL := FIND_S
YM(TEXT);END-IF;IF SYMBOL.TY = DN_VOID THEN DEFLIST := (TREE_NIL, TREE_NIL);ELSE DEFLIST := LIST(SYMBOL);END-IF;
WHILE NOT IS_EMPTY(DEFLIST) AND THEN HEAD(DEFLIST).TY = DN_TERMINAL LOOP DEFLIST := TAIL(DEFLIST);END-LOOP;IF
F IS_EMPTY(DEFLIST) THEN ERROR(SOURCEPOS, "UNDEFINED TERMINAL - " & TEXT);ELSE DI(LX_SRCPOS, HEAD(DEFLIST), SOURCEPOS);
END-IF;END MAKE_TERMINAL;BEGIN IF TOKEN = "%terminals" THEN ERROR(SOURCEPOS, "EXPECTING %terminals");RETURN;E
ND IF;AVANCER;MAKE_TERMINAL("%end");WHILE-TOKEN "/" = "%start" LOOP MAKE_TERMINAL(TOKEN);AVANCER;END-LOOP;E
IF TOKEN = "%start" THEN ERROR(SOURCEPOS, "EXPECTING %start");RETURN;END IF;AVANCER;SAUTER %start;-- GENE
RATE RULE: -- *SENTENCE* := <START_SYMBOL> *END* MAKE_RULE("%SENTENCE*");MAKE_ALTERNATIVE;MAKE_SYLLABLE(TOKEN);IF SYLLA
BLE.TY = DN_TERMINAL THEN ERROR(SOURCEPOS, "START SYMBOL CANNOT BE TERMINAL - " & TOKEN);END IF;MAKE_SYLLABLE("%end");SEM_S :=
MAKE(DN_SEM_S);DI(XD_SEM_INDEX, SEM_S, 0);LIST(SEM_S, (TREE_NIL, TREE_NIL));LIST(ALTERNATIVE, SYL_LIST);DI(XD_SEMANTIC
S, ALTERNATIVE, SEM_S);LIST(RULE, ALT_LIST);AVANCER;LIT LE %rules;IF TOKEN = "%rules" THEN ERROR(SOURCEPOS, "EX
PECTING %rules INSTEAD OF " & TOKEN);RETURN;END IF;AVANCER;SAUTER %rules;WHILE-TOKEN "/" = "%end" LOOP MAKE_RUL
E(TOKEN);AVANCER;IF TOKEN = "==" THEN IF AVANCER;ELSE ERROR(SOURCEPOS, "EXPECTING :: INSTEAD OF " & TOKEN);
END-IF;WHILE TOKEN = "==" AND THEN TOKEN = "==" AND THEN TOKEN = "==" AND THEN TOKEN = "==" AND THEN TOKEN = "==" LOOP
IF IF TOKEN = "empty" THEN MAKE_SYLLABLE(TOKEN);END-IF;AVANCER;END-LOOP;IF TOKEN = "==" THEN LISTE D'OPERA
TIONS SEMANTIQUES AVANCER;WHILE TOKEN = " " AND THEN TOKEN = " " AND THEN TOKEN = " " AND THEN TOKEN = " " AND THEN TOKEN = " " ARRET-DE LA LISTE SUR NOUVELLE ALTERNATIVE OU-FIN DE REGLE;MAKE_SEMA
NTICS_GET_TOKEN(TOKEN);INTEGER L'OPERATION SEMANTIQUE;END-LOOP;SEM_S := MAKE(DN_SEM_S);DI(XD_SEM
_INDEX, SEM_S, 0);LIST(SEM_S, SEMAN_LIST);IF NOT IS_EMPTY(SEMAN_LIST) THEN SEMAN_ALT_COUNT := SEMAN_ALT_COUNT + 1;END
IF;DI(XD_SEMANTICS, ALTERNATIVE, SEM_S);LIST(ALTERNATIVE, SYL_LIST);IF TOKEN = " " THEN ENCORE UNE ALTER
NATIVE, PASSER LE %AVANCER;END-IF;END-LOOP;LIST(RULE, ALT_LIST);LISTER LA REGLE;AVANCER;P
ASSER LE %GRAMMAR := MAKE(DN_RULES);LIST(GRAMMAR, RULE_LIST);USER_ROOT := MAKE(DN_USER_ROOT);DI(XD_SOURCE
NAME, USER_ROOT, STORE_TEXT(NOM_TEXTE));DI(XD_GRAMMAR, USER_ROOT, GRAMMAR);DI(XD_USER_ROOT, TREE_ROOT, USER_ROOT);END PROCESS_G
RAMMAR;
0010 AR;BEGIN OPEN (-IFILE, IN_FILE, ".../idl/" & "diana.idl");CONTIENT LA DESCRIPTION IDL DE LA GRAMMAIRE ADA83;CREATE(-OFILE, OUT_FIL
E, NOM_TEXTE & ".INTS.txt");CREATE IDL_TREE_FILE(NOM_TEXTE & ".lar");FICHER DES PAGES CONTENANT L ARBRE GRAMMAIRE-ADA83;SOURCE_LIS
T := (TREE_NIL, TREE_NIL);LOAD_DIANA;LOAD_TERMINALS;AVANCER;PUT_LINE("PROCESS_GRAMMAR");PROCESS_GRAMMAR;LIST(TREE_ROOT, SOURCE
_LIST);CLOSE(OFILE);CLOSE(IFILE);CLOSE IDL_TREE_FILE;INT_IO.PUT(SEMAN_COUNT, 0);PUT("SEM SYLS FOR");INT_IO.PUT(SEMAN_ALT
_COUNT, 0);PUT_LINE("ALTS");END READ_GRMR;
--|
```